

Introduction:

File Transfer Protocol (FTP) is used to transfer files between a client and a server over a network. It follows the client-server model where the client requests a file and the server provides it. Although real FTP uses dedicated protocols, similar functionality can be implemented using socket programming.

In this experiment, a basic FTP-like system is created using Java. The server reads a file and sends it as binary data, while the client receives and stores it locally.

Procedure (Code):

Server Code:

```
package com.server;
import java.io.*;
import java.net.*;

public class Dotnet {
    public static void main(String[] args) {
        try {
            ServerSocket server = new ServerSocket(7000);
            System.out.println("FTP Server is running on port 7000...");
            while (true) {
                Socket socket = server.accept();
                System.out.println("Client connected to FTP Server");

                File file = new File("Data.txt");
                if (!file.exists()) {
                    System.out.println("File not found!");
                    socket.close();
                    continue;
                }

                FileInputStream fis = new FileInputStream(file);
                BufferedOutputStream bos = new
                BufferedOutputStream(socket.getOutputStream());

                byte[] buffer = new byte[1024];
                int bytesRead;
```

```

        while ((bytesRead = fis.read(buffer)) > 0) {
            bos.write(buffer, 0, bytesRead);
        }

        bos.flush();
        fis.close();
        socket.close();

        System.out.println("File sent successfully!");
    }

} catch (Exception e) {
    System.out.println("FTP Server Error: " + e);
}
}
}

```

Output:

```

PS C:\Users\Fuad\Downloads> & 'C:\Pro
.Dotnet'
FTP Server is running on port 7000...
Client connected to FTP Server
File sent successfully!

```

Client Code:

```

package com;
import java.io.*;
import java.net.*;

```

```

public class ClientOG {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 7000);
            System.out.println("Connected to FTP Server");

```

```

                                BufferedInputStream    bis    =    new
BufferedInputStream(socket.getInputStream());
            FileOutputStream fos = new FileOutputStream("data_recive.txt");

            byte[] buffer = new byte[1024];
            int bytesRead;

            while ((bytesRead = bis.read(buffer)) > 0) {
                fos.write(buffer, 0, bytesRead);

```

```

    }

    fos.close();
    socket.close();

    System.out.println("File received and saved as data_recive.txt");

    } catch (Exception e) {
        System.out.println("FTP Client Error: " + e);
    }
}
}}

```

Output:

```

PS C:\Users\Fuad\Downloads> & 'C:\Program F
OG'
Connected to FTP Server
File received and saved as data_recive.txt
PS C:\Users\Fuad\Downloads>

```

Data.txt:

```

≡ Data.txt
1 Welcome to Dotnet's private FTP server of free Movies.

```

data_recive.txt:

```

≡ data_recive.txt
1 Welcome to Dotnet's private FTP server of free Movies.

```

Conclusion:

This experiment demonstrates a simple implementation of file transfer using socket programming. The server successfully transmits file data, and the client stores it locally. It provides a clear understanding of how file transfer mechanisms work in network communication systems.